UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Data representation and number systems

Answer: left -86 -> 1 1010110; right -68 -> 1 1000100; -86 - (-68) = -18 -> sign-magnitude 1 0010010

- *1 mark for converting both operands to sign-magnitude.*
- *1 mark for applying the sign rule for subtraction.*
- *1 mark for the correct magnitude operation.*
- *1 mark for the correct sign bit.*
- *1 mark for the final decimal and byte representation.*

QUESTION A2 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (7).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two advantages of SMP (Symmetric Multiprocessor) organization: (1) Performance/scalability — multiple identical processors share the same main memory and I/O, so several processes or threads can execute in parallel, increasing throughput and allowing the system to scale by adding processors. (2) Simplicity and cost-effectiveness — all processors are identical and share one copy of the OS and one memory space, so a single operating system manages the whole machine, giving easier programming, load balancing and better price/performance than equivalent multiple separate computers. (Other acceptable: fault tolerance/graceful degradation — if one processor fails the others continue; resource sharing — all processors share memory, I/O and peripherals, reducing duplication.)

(b) Cluster organization: a cluster is a set of independent, complete computers (nodes, each with its own processor and memory) interconnected by a high-speed network, which cooperate and appear to users as a single, unified system. The nodes work together to provide high availability and/or high performance, with software distributing work and coordinating them.

(c) One benefit of cluster organization: high availability / fault tolerance — if one node fails, the remaining nodes take over its work so the service continues (alternative acceptable benefit: high performance/scalability by adding nodes, or better price/performance using commodity machines).

Unofficial marking guide:

- *1 mark: first valid advantage of SMP (e.g. parallel processing/increased throughput/scalability by adding processors)*
- *1 mark: second valid advantage of SMP (e.g. shared memory and I/O / single OS / cost-effectiveness / graceful degradation)*
- *1 mark: cluster is a collection of independent computers/nodes*
- *1 mark: nodes are interconnected by a network and cooperate/appear as a single system (high availability or high performance)*
- *1 mark: one valid benefit of clustering (e.g. high availability/fault tolerance, scalability, price/performance)*

Model confidence: high.

Model note: Question text is complete; no figure required. Accept equivalent wording for advantages and benefits.

QUESTION A3 (5 marks)

Topic: Computer generations and history

Source: CSC429 (18).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The PC was first introduced in the fourth generation of computers. The fourth generation (roughly 1971 onwards) is characterised by microprocessors — a complete CPU on a single chip — which made it economically feasible to build small, low-cost computers for individual use. The first widely recognised personal computers (e.g. the Altair 8800 in 1975 and later the IBM PC in 1981) belong to this microprocessor-based fourth generation.

(b) The first commercially available computer was UNIVAC I (UNIVersal Automatic Computer I), delivered in 1951. (Note: ENIAC, 1946, was an earlier electronic computer but was not a commercial product; if the lecturer's notes treat ENIAC as the answer, credit should follow the course notes — see

© Hak Cipta Universiti Teknologi MARA

notes.)

(c) When the PC was first introduced, the interface available was a command-line interface (CLI): the user typed text commands at a prompt (e.g. in MS-DOS) and the system responded with text on the screen. Early machines also used switches/front panels and later text-based terminals; graphical user interfaces (GUIs) such as Windows became common only later.

(d) The key factor that makes a personal computer portable is the use of microprocessor/VLSI technology, which integrates the CPU and much of the system onto a few small, low-power chips. This drastically reduces the size, weight, heat dissipation and power consumption of the machine, so the whole computer can be built as a compact unit (laptop/portable) that runs from a battery. Supporting factors include lightweight flat-panel (LCD) displays and compact mass storage (hard disks, SSDs, flash), but the fundamental enabler is large-scale integration of components onto microchips.

Unofficial marking guide:

• 1 mark: (a) Fourth generation (microprocessor-based generation).

• 1 mark: (b) UNIVAC I (accept ENIAC only if the course notes define it as the first commercial computer).

• 1 mark: (c) Command-line interface (text-based interface); accept front-panel switches/text terminal for very early PCs.

• 1 mark: (d) Identifies microprocessor/VLSI integration as the key factor reducing size, weight and power consumption.

• 1 mark: (d) Explains the consequence — compact, low-power, battery-operable unit (with mention of supporting factors such as LCD display or compact storage).

Model confidence: medium.

Model note: Part (b) is ambiguous: UNIVAC I is the standard 'first commercially available computer', but some course notes name ENIAC; the answer states the assumption. Part (c) depends on how 'first introduced' is defined (Altair 8800 used switches; IBM PC used a CLI/PC-DOS), so the CLI answer is given with the early-hardware caveat.

QUESTION A4 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The addressing technique illustrated in Figure 2 is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective address of the operand in memory, so EA = A; the operand is fetched directly from memory location A.

(b) How it is used: The instruction is fetched and its opcode decoded. The address field A is taken as the effective address with no further calculation or indirection. The CPU places A on the memory address bus, performs a memory read, and the operand stored at location A is transferred to the CPU (into a register or the ALU) for use by the instruction. For example, if the instruction is LOAD A, the contents of memory location A are loaded into the accumulator. The operand is thus accessed in a single memory reference beyond the instruction fetch.

(c) Advantage: It is simple and fast — no address calculation or extra memory access is needed to determine the effective address, so only one operand memory access is required. Disadvantage: The address field must be large enough to hold a full memory address, limiting the number of bits available for the opcode/instruction range, and the address is fixed at assembly time, so the technique offers limited flexibility (no easy relocation or support for large address spaces).

Unofficial marking guide:

• 1 mark: correctly names the technique as direct addressing (absolute addressing).

• 1 mark: states that the address field A is the effective address (EA = A) with no indirection.

• 1 mark: explains the operand is fetched directly from memory location A in a single memory access for use by the instruction.

• 1 mark: gives one valid advantage (e.g. simple/fast, no address calculation, single operand memory access).

• 1 mark: gives one valid disadvantage (e.g. limited address range/opcode bits, fixed address, inflexibility).

Model confidence: high.

Model note: Figure 2 is inferred from the extracted text (Instruction: Opcode | Address A; Memory: Operand) to depict direct addressing. If the actual figure shows an extra indirection (address field pointing to a memory word that holds the operand address), the answer would instead be indirect addressing; the text as given supports direct addressing.

QUESTION A5 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (15).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two bus types available in a computer system: (1) the system bus (also called the front-side/memory bus), which connects the CPU to main memory and I/O via address, data and control lines; and (2) the I/O bus (expansion/peripheral bus, e.g. PCI/PCIe or USB), which connects peripheral devices and controllers to the system. Other acceptable answers include the address bus, data bus and control bus (the three functional buses of the system bus).

(b) Two differentiating characteristics of system interconnection versus CPU interconnection:
1. Scope/function: System interconnection links the major components of the whole computer (CPU, memory, I/O controllers) so they can exchange data, whereas CPU interconnection is internal to the processor, linking its functional units (registers, ALU, control unit, cache) to coordinate instruction execution.
2. Speed/bandwidth and technology: CPU interconnection is designed for very high speed and low latency using on-chip buses or point-to-point links, while system interconnection is slower and must support multiple heterogeneous devices and arbitration, typically using a shared bus or switched interconnect. (A third valid contrast: system interconnection must handle arbitration among many bus masters; CPU interconnection is tightly controlled by the CPU's own control unit.)

(c) Data transfer timing types:
- Synchronous timing: all events are governed by a common clock; the sender and receiver transfer data at fixed, clock-defined intervals. It is simple and fast for devices of similar speed but inefficient when device speeds differ.
- Asynchronous timing: no common clock; the transfer is coordinated by handshaking signals (e.g. request/acknowledge or strobe lines), so each device can operate at its own speed. It is flexible but adds handshake overhead.
- Semi-synchronous (synchronous with wait states): a clocked scheme that allows a slow device to insert wait states to extend the cycle, combining the simplicity of synchronous timing with some flexibility for slower devices.

Unofficial marking guide:

- *1 mark: correctly naming the system bus (or address/data/control bus)*
- *1 mark: correctly naming a second bus type such as the I/O/expansion bus*
- *1 mark: first valid differentiating characteristic of system vs CPU interconnection*
- *1 mark: explanation/elaboration of that first characteristic*
- *1 mark: second valid differentiating characteristic*
- *1 mark: explanation/elaboration of that second characteristic*
- *1 mark: correct description of synchronous timing*
- *1 mark: correct description of asynchronous timing*
- *1 mark: correct description of semi-synchronous timing (or a third valid timing type)*
- *1 mark: accurate use of terminology (clock, handshaking, wait states) across the timing explanation*

Model confidence: high.

Model note: Question text has minor OCR/formatting issues ('buses types', line break in part b); interpreted conservatively. Part (b) asks for TWO characteristics, so the answer gives two contrasts with a third as optional. Part (c) assumes the standard three timing schemes (synchronous, asynchronous, semi-synchronous) taught in CSC429.

QUESTION A6 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (10).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Four storage access methods, each with an example:
1. Sequential access — data is read in a fixed linear order from start to end; to reach a record you must pass through all preceding ones. Example: magnetic tape.
2. Direct access — data is organised into blocks/records that can be jumped to directly (by block address), but within a block access is sequential. Example: magnetic disk (hard disk drive).
3. Random access — any location can be selected and accessed in approximately the same time, independent of position. Example: main memory (RAM / semiconductor memory).
4. Associative access — data is located by matching its content (a key/tag) rather than by address, and comparison is done in parallel across all locations. Example: cache memory (associative cache / content-addressable memory).

© Hak Cipta Universiti Teknologi MARA

(b) With 11 address bits, the number of addressable memory locations is 2^11 = 2048 locations (2 K locations).

(c) 2 MB = 2 × 2^20 bytes = 2^21 bytes. Each byte needs one address, so the number of addressable locations = 2^21, requiring 21 address bits. (Equivalently, log2(2 MB) = log2(2^21) = 21 bits.)

Unofficial marking guide:

• 1 mark: correct naming of sequential access method
• 1 mark: correct example for sequential access (magnetic tape)
• 1 mark: correct naming of direct access method
• 1 mark: correct example for direct access (magnetic disk)
• 1 mark: correct naming of random access method
• 1 mark: correct example for random access (RAM/main memory)
• 1 mark: correct naming of associative access method
• 1 mark: correct example for associative access (cache/CAM)
• 1 mark: 2^11 = 2048 memory locations (accept 2K)
• 1 mark: 2 MB = 2^21 bytes and hence 21 address bits

Model confidence: high.

Model note: Part (a) awards 6 marks for four methods; the marking guide gives 1.5 marks per method (0.75 name + 0.75 example), rounded here to 1 mark per name and 1 mark per example across the four methods. If the lecturer treats 'direct' and 'random' as one category, accept 'direct/random access' with disk or RAM as the example. Part (c) assumes byte-addressable memory, which is the standard convention.

QUESTION A7 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (13).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Definition of interrupt: An interrupt is a signal (either generated by hardware or by software) sent to the CPU that temporarily suspends the currently executing program, causes the processor to save its current state (program counter and registers), and transfers control to a special service routine — the interrupt service routine (ISR) / interrupt handler — which deals with the event; when the handler finishes, the saved state is restored and the interrupted program resumes. In short, it is a mechanism by which a device or program requests the CPU's attention so that it can be serviced.

(b) Interrupt versus direct manipulation access (DMA):
- Meaning: An interrupt is a signal that diverts the CPU to run a service routine for an event; DMA (direct memory access) is a technique in which a dedicated DMA controller transfers data directly between an I/O device and main memory without the CPU handling each word.
- Role of CPU: With interrupts the CPU stops its current work and executes the ISR for every event/transfer unit; with DMA the CPU only initialises the transfer (sets up the DMA controller with source, destination and count) and is informed by a single interrupt when the whole block is complete, so the CPU is largely free during the transfer.
- Data transfer granularity: Interrupt-driven I/O transfers data one word/byte at a time under program control; DMA transfers whole blocks of data.
- Speed/efficiency: Interrupt-driven I/O is slower and imposes overhead (context save/restore, ISR execution) for each item; DMA is much faster for large transfers and gives higher throughput because it bypasses the CPU per-word overhead.
- Usage: Interrupts are used for low-volume or urgent events (keyboard, mouse, error conditions); DMA is used for high-volume transfers (disk, network, graphics).
- Common point: Both free the CPU from continuously polling the device, and DMA normally signals completion by raising an interrupt.

(c)(i) Types of interrupt:
- Figure 2(a) — "Are you sure you want to cancel this request?": This is a software interrupt (a synchronous/trap-type interrupt generated by the running program, i.e. an internal interrupt). Justification: it is produced deliberately by the application/OS in response to a user action (a confirmation dialog for cancelling a request), not by any hardware device; it is predictable and tied to the instruction being executed.
- Figure 2(b) — "There is a hardware event failure in this computer. Please reboot this computer now.": This is a hardware interrupt (an asynchronous/external interrupt, here effectively a hardware failure/error condition). Justification: the message explicitly reports a hardware event failure, i.e. the signal originates

© Hak Cipta Universiti Teknologi MARA

from hardware (e.g. a device or the motherboard raising an error), it is asynchronous to the running program and must be serviced immediately, forcing a reboot.

(c)(ii) Nested/priority interrupt: Because interrupt B has higher priority than interrupt A, the CPU will suspend (pre-empt) the currently executing interrupt routine A — saving its state (PC and registers) on the stack — and transfer control to the ISR for interrupt B. After B's routine completes, the saved state of A is restored and routine A resumes from the point at which it was interrupted. This is nested interrupt handling (interrupt A is temporarily delayed, not lost).

Unofficial marking guide:

• 1 mark: (a) correct definition of an interrupt as a signal that suspends the current program and transfers control to a service routine (with state save/restore).

• 1 mark: (b) correct meaning of interrupt and of DMA (direct memory access).

• 1 mark: (b) CPU involvement — CPU services each event for interrupts vs CPU only sets up and is notified at completion for DMA.

• 1 mark: (b) data granularity/speed — word-at-a-time, slower interrupt I/O vs block transfer, faster DMA.

• 1 mark: (b) appropriate usage/example contrast (e.g. keyboard interrupt vs disk DMA) or valid common point.

• 2 marks: (c)(i) Figure 2(a) identified as software/internal interrupt with justification (generated by program/user action, not hardware).

• 2 marks: (c)(i) Figure 2(b) identified as hardware/external interrupt with justification (hardware event failure, asynchronous, forces reboot).

• 1 mark: (c)(ii) correct explanation that routine A is suspended/pre-empted, B is serviced first, then A resumes (nested interrupt).

Model confidence: high.

Model note: The question writes 'direct manipulation access'; this is taken to mean direct memory access (DMA), the standard I/O topic paired with interrupts. Figure 2 is only available as extracted text; the identification relies on the wording of the two pop-up messages rather than any graphical detail.

QUESTION A8 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (2).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two types of keyboard characters [4 marks]

1. Printable (alphanumeric/graphic) characters: These are the ordinary characters that produce a visible symbol on the display or printer, such as letters (A–Z, a–z), digits (0–9), punctuation marks (., ; ? etc.) and special symbols (@, #, $). When a key is pressed the keyboard's encoder generates the corresponding character code (e.g. its ASCII value) which is sent to the I/O module and then to the processor for display or processing. They are the characters used to build text and data.

2. Control (non-printable) characters: These do not produce a visible symbol but instead cause an action or control function to be performed, e.g. Enter/Carriage Return (CR), Line Feed (LF), Tab, Backspace, Escape, Delete and cursor-control keys. They are represented by control codes (e.g. ASCII codes 0–31) and are interpreted by the software/driver to move the cursor, start a new line, delete a character or signal the end of input. (Accept equivalent pairing such as alphanumeric vs. special/function keys, provided each type is explained and distinguished.)

(b) Three categories of input/output module [6 marks]

1. Human-readable / human-machine interface modules: These convert between human-oriented and machine-oriented representations, i.e. they connect the computer to the user. Example: keyboard and monitor (VDU), or a printer — the keyboard converts key presses into character codes and the display converts codes into visible text.

2. Machine-readable modules: These interface the computer to equipment that stores or reads data in machine-suitable form, without human intervention. Example: a magnetic disk drive controller, magnetic tape unit, or a barcode/OCR reader feeding data directly into the system.

3. Communication modules: These allow the computer to exchange data with remote devices or other computers over a communication link, performing serial/parallel conversion and protocol handling. Example: a network interface card (NIC), modem, or a serial/parallel communication port (UART).

(Each category needs a description plus a valid example to earn full credit.)

Unofficial marking guide:

• 1 mark: correctly identifies the first type of keyboard character (e.g. printable/alphanumeric)

© Hak Cipta Universiti Teknologi MARA

• 1 mark: explains that it produces a visible symbol and is encoded as a character code (e.g. ASCII) sent to the processor
• 1 mark: correctly identifies the second type (e.g. control/non-printable or special/function keys)
• 1 mark: explains that it performs an action/control function rather than displaying a symbol, with an example (Enter, Tab, Backspace, etc.)
• 1 mark: correctly names the first I/O module category (human-readable)
• 1 mark: describes it and gives an appropriate example (keyboard, monitor, printer)
• 1 mark: correctly names the second category (machine-readable)
• 1 mark: describes it and gives an appropriate example (disk controller, tape, barcode reader)
• 1 mark: correctly names the third category (communication)
• 1 mark: describes it and gives an appropriate example (NIC, modem, serial port)
Model confidence: medium.
Model note: The question wording 'types of keyboard characters' is interpreted as printable vs. control/non-printable characters; alternative valid pairings (alphanumeric vs. special/function keys) are accepted. The I/O module categories follow the standard Stallings classification (human-readable, machine-readable, communication); other reasonable classifications with valid examples are accepted. No figure or table is required.

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation
Source: CSC429 (6).pdf, page(s) 3
Unofficial model answer — generated by an AI model, not an official UTM marking scheme.
(a) Computer architecture refers to the attributes of a computer system that are visible to a programmer, i.e. the conceptual design and functional behaviour of the system. It includes the instruction set, data types, addressing modes, registers visible to the programmer, and I/O mechanisms. Architecture answers the question 'what does the computer do?' from the programmer's perspective. Computer organization refers to the operational units and their interconnections that realize the architectural specifications. It deals with the physical implementation details: control signals, memory technology, clock frequency, bus structure, and the way components are arranged. Organization answers 'how is the architecture implemented?' and can change without affecting the programmer-visible architecture (e.g. changing cache size or clock speed). In summary, architecture is the programmer-visible functional view, while organization is the internal physical realization.

(b) Three CPU components and their functions:
1. Control Unit (CU): It fetches instructions, decodes them, and generates the timing and control signals that direct the operation of the other CPU components (ALU, registers, memory, I/O). It coordinates the execution of instructions by sequencing micro-operations.
2. Arithmetic Logic Unit (ALU): It performs arithmetic operations (such as addition, subtraction, increment, decrement) and logical operations (such as AND, OR, NOT, XOR, shift, rotate) on data held in registers. It also sets condition flags (zero, carry, sign, overflow) used for decision making.
3. Registers (e.g. Program Counter, Instruction Register, Accumulator, General-purpose registers): They provide fast, temporary storage inside the CPU. The Program Counter holds the address of the next instruction; the Instruction Register holds the current instruction during execution; the Accumulator/general registers hold operands and results during processing. Registers reduce memory access time and support instruction execution.

Unofficial marking guide:
• 1 mark: correct definition of architecture (programmer-visible attributes, instruction set, etc.)
• 1 mark: correct definition of organization (physical implementation, interconnections, control signals, etc.)
• 1 mark: clear contrast between the two (e.g. architecture = what, organization = how; organization can change without affecting architecture)
• 1 mark: appropriate example illustrating the difference (e.g. same architecture with different cache sizes/clock speeds)
• 1 mark: correct identification of first CPU component (e.g. Control Unit)
• 1 mark: correct explanation of first component's function
• 1 mark: correct identification of second CPU component (e.g. ALU)
• 1 mark: correct explanation of second component's function
• 1 mark: correct identification of third CPU component (e.g. registers)
• 1 mark: correct explanation of third component's function
Model confidence: high.
Model note: The question text is clear; no OCR corrections needed. The answer assumes standard CPU components (Control Unit, ALU, registers) are acceptable; other valid components (e.g. cache, bus interface) would also be acceptable if explained correctly.

© Hak Cipta Universiti Teknologi MARA

QUESTION A10 **(10 marks)**

Topic: Data representation and number systems

a) Answer: left 50 -> 0 0110010; right -64 -> 1 1000000; 50 - (-64) = 114 -> sign-magnitude 0 1110010

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

b) Answer: left 50 -> 1's 00110010, 2's 00110010; right -64 -> 1's 10111111, 2's 11000000; 50 - (-64) = 114 -> 1's 01110010, 2's 01110010

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*
- *1 mark for converting the result back to decimal.*

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (21).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 5 reads z-A+c2/B*D-3A. The extracted text is garbled; the standard form of this classic question is taken to be z = A + (C × 2) / (B × D) − 3A, i.e. z = A + C*2/B*D - 3A. If the figure actually shows a different arrangement (e.g. C^2 or C*2), the same instruction sequences apply with the operands reordered; the structure of the answer is unchanged. Registers A, B, C, D are fixed; x, y, z are scratch; result must end in z.

Zero-address (stack) program. All operands are pushed onto the stack; operations pop their operands and push the result. Evaluate left-to-right with the usual precedence (× and / before + and −):

```
PUSH A        ; stack: A
PUSH C
PUSH 2
MUL           ; C*2
PUSH B
PUSH D
MUL           ; B*D
DIV           ; (C*2)/(B*D)
ADD           ; A + (C*2)/(B*D)
PUSH 3
PUSH A
MUL           ; 3A
SUB           ; A + (C*2)/(B*D) - 3A
POP z         ; store final answer in z
```

One-address program. One operand is implicit in the accumulator; the named operand is the memory/register operand. Using x and y as temporaries:

```
LOAD C
MUL 2         ; ACC = C*2
STOR x        ; x = C*2
LOAD B
MUL D         ; ACC = B*D
STOR y        ; y = B*D
LOAD x
DIV y         ; ACC = (C*2)/(B*D)
ADD A         ; ACC = A + (C*2)/(B*D)
STOR x        ; save partial result in x
LOAD 3
MUL A         ; ACC = 3A
STOR y        ; y = 3A
LOAD x
SUB y         ; ACC = A + (C*2)/(B*D) - 3A
STOR z        ; final answer in z
```

Three-address program. Each instruction is OP (dest, src1, src2); temporaries x and y hold intermediate results:

```
MUL (x, C, 2)      ; x = C*2
MUL (y, B, D)      ; y = B*D
DIV (x, x, y)      ; x = (C*2)/(B*D)
ADD (x, A, x)      ; x = A + (C*2)/(B*D)
MUL (y, 3, A)      ; y = 3A
SUB (z, x, y)      ; z = A + (C*2)/(B*D) - 3A
```

In all three versions the final value is placed in register z, and the fixed registers A, B, C, D are never overwritten (only x and y are used as scratch).

© Hak Cipta Universiti Teknologi MARA

Unofficial marking guide:

• 4 marks: correct zero-address program — correct PUSH/POP ordering, correct use of MUL/DIV/ADD/SUB, result POPped into z

• 4 marks: correct one-address program — correct LOAD/STOR/MUL/DIV/ADD/SUB with accumulator, temporaries x and y used correctly, result STORed to z

• 4 marks: correct three-address program — correct three-operand MUL/DIV/ADD/SUB form, temporaries used, result written to z

• 4 marks: correct evaluation order/precedence (× and ÷ before + and −), correct handling of the 3A term, and no corruption of fixed registers A, B, C, D

• 4 marks: overall correctness and consistency across the three programs (same computed expression, final answer in z in every case)

Model confidence: medium.

*Model note: The equation in Figure 5 is OCR-garbled ('z-A+c2/B*D-3A'); I conservatively interpreted it as z = A + C*2/(B*D) − 3A, the standard form of this question. If the figure shows C² or a different grouping, the operand order changes but the instruction structure and marking allocation remain valid. The exact register/operand syntax of the three-address form is taken from Table 1 as OP(p,q,r).*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA